Session 5: Satellite Constellations and Coverage Analysis

Task 1: Constellation Coverage Theory & Requirements.

Trade-off Analysis:

- **Altitude** (h): Higher orbital altitudes increase the satellite footprint radius (R_{fp}) and Earth central angle (λ), expanding instant line-of-sight coverage per satellite and significantly reducing the total satellite count required for global coverage. However, higher altitudes increase signal path loss (free-space path loss $\propto d^2$, propagation latency, and radiation exposure (van Allen belts).
- **Inclination** (i): Determines the maximum latitude band receiving continuous or high-density coverage. Polar/near-polar inclinations $(i \approx 85^o - 90^o)$ provide full global coverage including the poles, whereas inclined Walker constellations concentrate coverage around mid-to-low latitude populated zones.
- **Minimum Elevation Mask Angle** (ε): Defines the minimum clearance angle above the local horizon required for reliable communication. A higher elevation mask $(\varepsilon = 10^o - 15^o)$ mitigates ground terrain blockage and atmospheric interference but shrinks the effective footprint size, requiring more satellites to prevent coverage gaps.
- **Satellite Count** (T): Increasing total satellites improves revisit frequency and spatial coverage continuity, but linearly scales capital expenditure (CapEx), launch manifest complexity, and constellation management overhead

Mission Coverage Requirement:

- **Objective**: Achieve continuous continuous global low-latency communication service for populated latitudes $(|lat| \leq 70^o)$ with a maximum acceptable revisit gap under 15 minutes and mean revisit gap below 5 minutes.

Task 2: Architectural Design & Hand Calculations

Use $R_E = 6378.137\ km$ and minimum elevation mask angle $\varepsilon = 10^o$:

1. **Spherical Geometry Formulas:**

- **Nadir Angle ($\boldsymbol{\eta}$):**

$$\sin\eta = \frac{R_E}{R_E + h}\cos\varepsilon$$

- **Earth Central Angle ($\boldsymbol{\lambda}$):**

$$\lambda = 90^o - \varepsilon - \eta$$

- **Footprint Radius ($\boldsymbol{R_{fp}}$):**

$$R_{fp} = R_E \cdot \lambda_{rad}$$

2. **Design A: Low-Altitude / High-Count Shell** ($h = 550\ km$)

- **Geometry Calculations:**

$$\sin\eta_A = \frac{6378.137}{6378.137 + 550}\cos(10^o) = 0.9066 \Rightarrow \eta_A = 65.04^o$$

$$\lambda_A = 90^o - 10^o - 65.04^o = 14.96^o$$

$$R_{fp,A} = 6378.137 \times \left(14.96^o \times \frac{\pi}{180^o}\right) = 1665\ km$$

- **Walker Delta Notation:** $80^o\!:120/10/1$
  - **Total Satellites ($\boldsymbol{T}$):** 120
  - **Orbital Planes ($\boldsymbol{P}$):** 10
  - **Satellites per Plane ($\boldsymbol{S}$):** 12
  - **Inter-plane Phase Factor ($\boldsymbol{F}$):** 1
  - **Plane Spacing:** $\Delta\Omega = \frac{360^o}{P} = 36^o$

3. **Design B: Higher-Altitude / Lower-Count Shell** ($h = 1200\ km$)

- **Geometry Calculations:**

$$\sin\eta_A = \frac{6378.137}{6378.137 + 1200}\cos(10^o) = 0.8289 \Rightarrow \eta_A = 55.98^o$$

$$\lambda_A = 90^o - 10^o - 55.98^o = 24.02^o$$

$$R_{fp,A} = 6378.137 \times \left(24.02^o \times \frac{\pi}{180^o}\right) = 2673\ km$$

- **Walker Delta Notation:** $87^o\!:48/8/2$
  - **Total Satellites ($\boldsymbol{T}$):** 48
  - **Orbital Planes ($\boldsymbol{P}$):** 8
  - **Satellites per Plane ($\boldsymbol{S}$):** 6
  - **Inter-plane Phase Factor ($\boldsymbol{F}$):** 2

- **Plane Spacing:** $\Delta\Omega = \frac{360^o}{P} = 45^o$

Task 3: Simulation Script

```
import numpy as np

# ------------------------------------------------------------------------------
# 1. CONSTANTS & GEOMETRY FUNCTIONS
# ------------------------------------------------------------------------------
R_E = 6378.137       # Earth radius in km (WGS84 / Standard)
MU = 398600.4418     # Standard gravitational parameter (km^3/s^2)
OMEGA_E = 7.292115e-5  # Earth rotation rate (rad/s)
SPEED_OF_LIGHT = 299792.458 # km/s

def compute_geometry(h_km, eps_deg):
    """Computes basic coverage geometry parameters."""
    r = R_E + h_km
    eps = np.radians(eps_deg)

    # Nadir angle at satellite
    eta = np.arcsin((R_E / r) * np.cos(eps))

    # Earth central angle (lambda)
    lam = np.pi / 2 - eps - eta

    # Slant range at edge of coverage
    rho = r * (np.sin(lam) / np.cos(eps))

    # Footprint radius (km) & coverage fraction f
    footprint_km = R_E * lam
    f_cov = (1 - np.cos(lam)) / 2.0

    # Orbital period (seconds)
    T_sec = 2 * np.pi * np.sqrt(r**3 / MU)

    # Max access duration per overhead pass (minutes)
    t_access_max = (lam / np.pi) * T_sec / 60.0

    # One-way edge latency (ms)
    edge_latency_ms = (rho / SPEED_OF_LIGHT) * 1000.0

    return {
        'r': r,
        'lambda_rad': lam,
        'lambda_deg': np.degrees(lam),
        'footprint_km': footprint_km,
        'f_cov': f_cov,
        'T_sec': T_sec,
        't_access_max_min': t_access_max,
        'edge_latency_ms': edge_latency_ms
    }

# ------------------------------------------------------------------------------
# 2. CONSTELLATION PROPAGATION & GRID COVERAGE SIMULATOR
# ------------------------------------------------------------------------------
def generate_walker_constellation(inc_deg, T_sats, P_planes, F_phase, a_km):
    """Generates initial orbital elements for Walker Delta/Star constellation."""
    inc = np.radians(inc_deg)
    sats_per_plane = T_sats // P_planes

    delta_RAAN = 2 * np.pi / P_planes  # Delta spacing
    delta_u = 2 * np.pi / sats_per_plane
    delta_phase = F_phase * (2 * np.pi / T_sats)

    satellites = []
    for p in range(P_planes):
        raan = p * delta_RAAN
        for s in range(sats_per_plane):
```

```
            u0 = s * delta_u + p * delta_phase
            satellites.append({'inc': inc, 'raan0': raan, 'u0': u0, 'a': a_km})

    return satellites

def propagate_satellites_ecef(satellites, t_array):
    """Propagates all satellites over time array and converts to ECEF unit vectors."""
    n_times = len(t_array)
    n_sats = len(satellites)

    # Matrix of sat unit vectors in ECEF: [n_times, n_sats, 3]
    sat_vecs = np.zeros((n_times, n_sats, 3))

    for i, sat in enumerate(satellites):
        n = np.sqrt(MU / sat['a']**3) # Mean motion (rad/s)
        inc = sat['inc']

        for j, t in enumerate(t_array):
            u = sat['u0'] + n * t
            raan_eci = sat['raan0'] # Simplified Keplerian
            raan_ecef = raan_eci - OMEGA_E * t

            # Position in ECEF unit vector
            x = np.cos(raan_ecef) * np.cos(u) - np.sin(raan_ecef) * np.sin(u) * np.cos(inc)
            y = np.sin(raan_ecef) * np.cos(u) + np.cos(raan_ecef) * np.sin(u) * np.cos(inc)
            z = np.sin(u) * np.sin(inc)

            sat_vecs[j, i] = [x, y, z]

    return sat_vecs

def create_global_grid(grid_step_deg=5.0):
    """Creates an area-weighted global lat-lon grid with ECEF unit vectors."""
    lats = np.arange(-90 + grid_step_deg/2, 90, grid_step_deg)
    lons = np.arange(-180 + grid_step_deg/2, 180, grid_step_deg)

    grid_units = []
    weights = []

    for lat in lats:
        lat_rad = np.radians(lat)
        w = np.cos(lat_rad) # Area weight
        for lon in lons:
            lon_rad = np.radians(lon)
            x = np.cos(lat_rad) * np.cos(lon_rad)
            y = np.cos(lat_rad) * np.sin(lon_rad)
            z = np.sin(lat_rad)
            grid_units.append([x, y, z])
            weights.append(w)

    return np.array(grid_units), np.array(weights)

def  run_simulation(design_name,  h_km,  inc_deg,  T_sats,  P_planes,  F_phase,  eps_deg=10.0,
dt_min=2.0):
    """Runs a 24-hour simulation for a specific design architecture."""
    # Geometry calculations
    geo = compute_geometry(h_km, eps_deg)

    # Grid setup
    grid_vecs, weights = create_global_grid(grid_step_deg=4.0)
    weights /= np.sum(weights) # Normalize area weights
    n_grid = len(grid_vecs)

    # Time vector (24 hours)
    t_array = np.arange(0, 86400, dt_min * 60.0)
    n_times = len(t_array)

    # Constellation setup & propagation
    sats = generate_walker_constellation(inc_deg, T_sats, P_planes, F_phase, geo['r'])
    sat_vecs = propagate_satellites_ecef(sats, t_array)
```

```
    # Threshold for central angle
    cos_lam_thresh = np.cos(geo['lambda_rad'])

    # Coverage tracking matrix: [n_times, n_grid] -> True/False
    coverage_matrix = np.zeros((n_times, n_grid), dtype=bool)

    for j in range(n_times):
        # Dot product between all sats and all grid points: [n_sats, n_grid]
        dots = np.dot(sat_vecs[j], grid_vecs.T)
        # Point is covered if any sat dot product >= cos(lambda)
        coverage_matrix[j] = np.any(dots >= cos_lam_thresh, axis=0)

    # ------------------------------------------------
    # Calculate Figures of Merit (FOM)
    # ------------------------------------------------
    # 1. Percent coverage (Area-weighted 24h average)
    point_coverage = np.mean(coverage_matrix, axis=0)
    percent_coverage = np.sum(point_coverage * weights) * 100.0

    # 2. Revisit gaps calculation per grid point
    mean_gaps = []
    max_gaps = []

    for g in range(n_grid):
        timeline = coverage_matrix[:, g]
        if np.all(timeline): # Continuously covered
            mean_gaps.append(0.0)
            max_gaps.append(0.0)
        else:
            # Find gap durations
            gap_lengths = []
            curr_gap = 0
            for vis in timeline:
                if not vis:
                    curr_gap += dt_min
                else:
                    if curr_gap > 0:
                        gap_lengths.append(curr_gap)
                        curr_gap = 0
            if curr_gap > 0:
                gap_lengths.append(curr_gap)

            if len(gap_lengths) == 0:
                mean_gaps.append(0.0)
                max_gaps.append(0.0)
            else:
                mean_gaps.append(np.mean(gap_lengths))
                max_gaps.append(np.max(gap_lengths))

    mean_revisit = np.sum(np.array(mean_gaps) * weights)
    worst_gap = np.max(max_gaps)

    return {
        'Design': design_name,
        'Altitude (km)': h_km,
        'Inclination (deg)': inc_deg,
        'Walker': f"{T_sats}/{P_planes}/{F_phase}",
        'Satellites': T_sats,
        'Central Angle (deg)': round(geo['lambda_deg'], 1),
        'Footprint Radius (km)': int(round(geo['footprint_km'], -1)),
        'Coverage (%)': round(percent_coverage, 1),
        'Mean Revisit (min)': round(mean_revisit, 1),
        'Worst Gap (min)': int(round(worst_gap)),
        'Access/pass max (min)': round(geo['t_access_max_min'], 1),
        'Edge Latency (ms)': round(geo['edge_latency_ms'], 1)
    }

# ------------------------------------------------------------------------------
# 3. MAIN EXECUTION (RUNNING DESIGNS A & B)
# ------------------------------------------------------------------------------
if   name   == "  main  ":
```

```
    print("Simulating Constellation Architectures (24-Hour Global Grid)...\n")

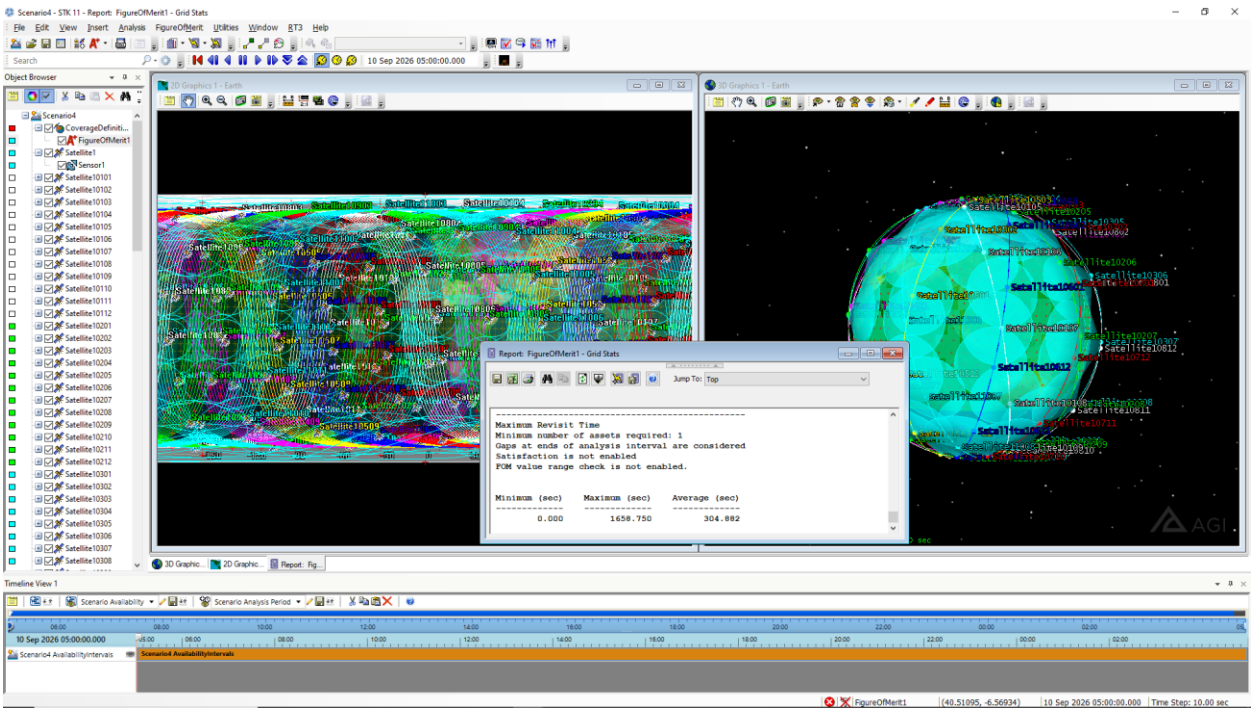
    # Run Design A (Low shell: 550 km, 80 deg, 120/10/1)
    res_A = run_simulation("A: Low shell", h_km=550, inc_deg=80, T_sats=120, P_planes=10,
F_phase=1)

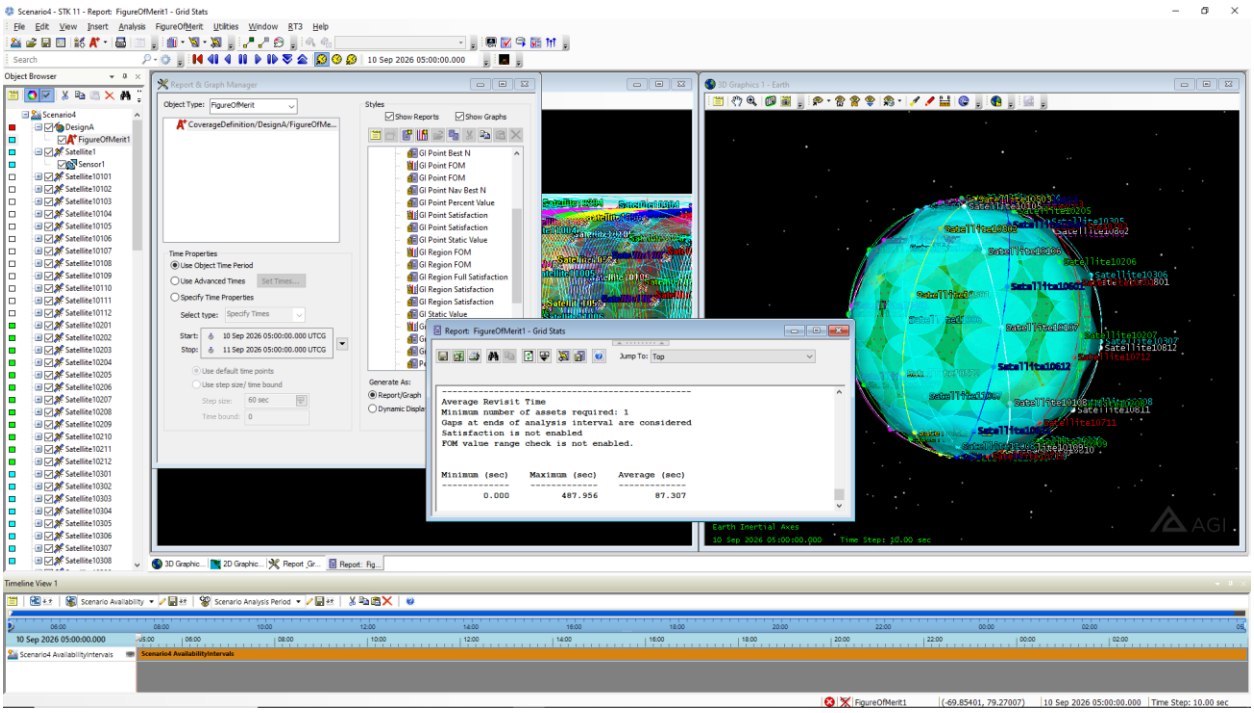
    # Run Design B (Mid shell: 1200 km, 87 deg, 48/8/2)
    res_B = run_simulation("B: Mid shell", h_km=1200, inc_deg=87, T_sats=48, P_planes=8,
F_phase=2)

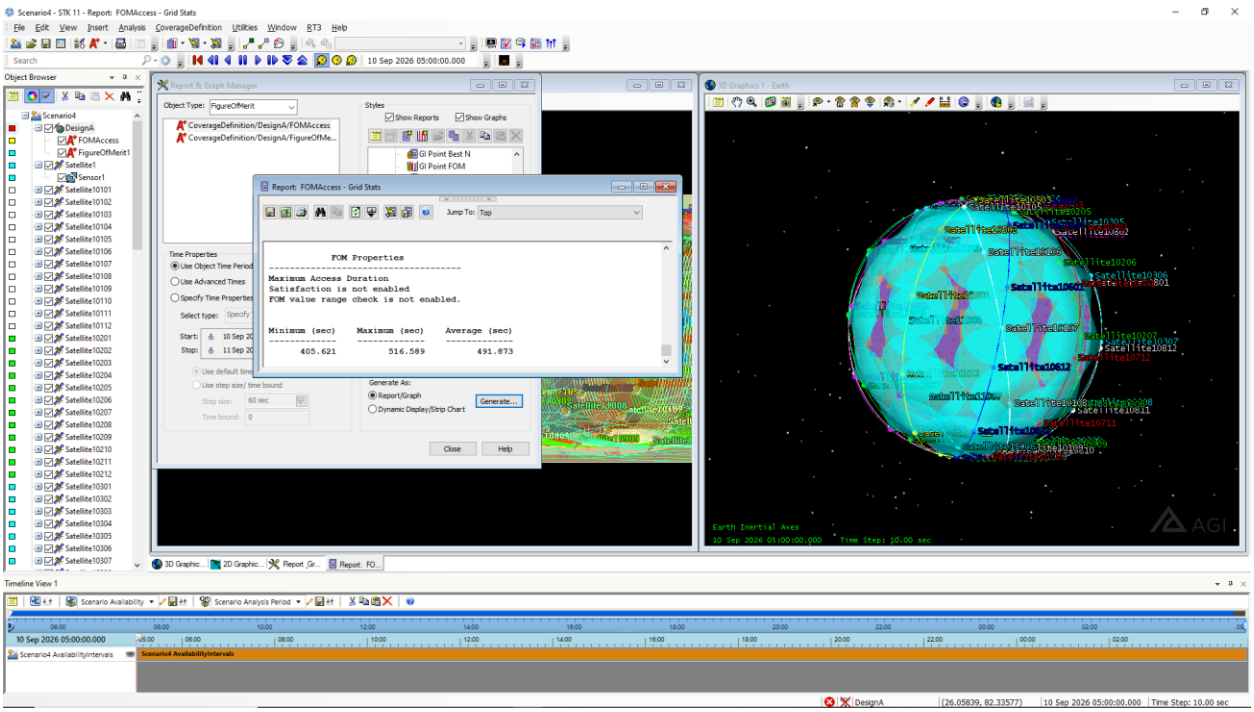
    # Print Comparison Table
    headers = list(res_A.keys())
    print(f"{'Metric':<25} | {'Design A (550 km)':<20} | {'Design B (1200 km)':<20}")
    print("-" * 71)
    for k in headers:
        if k == 'Design': continue
        val_a = str(res_A[k])
        val_b = str(res_B[k])
        print(f"{k:<25} | {val_a:<20} | {val_b:<20}")
```

Task 4: STK

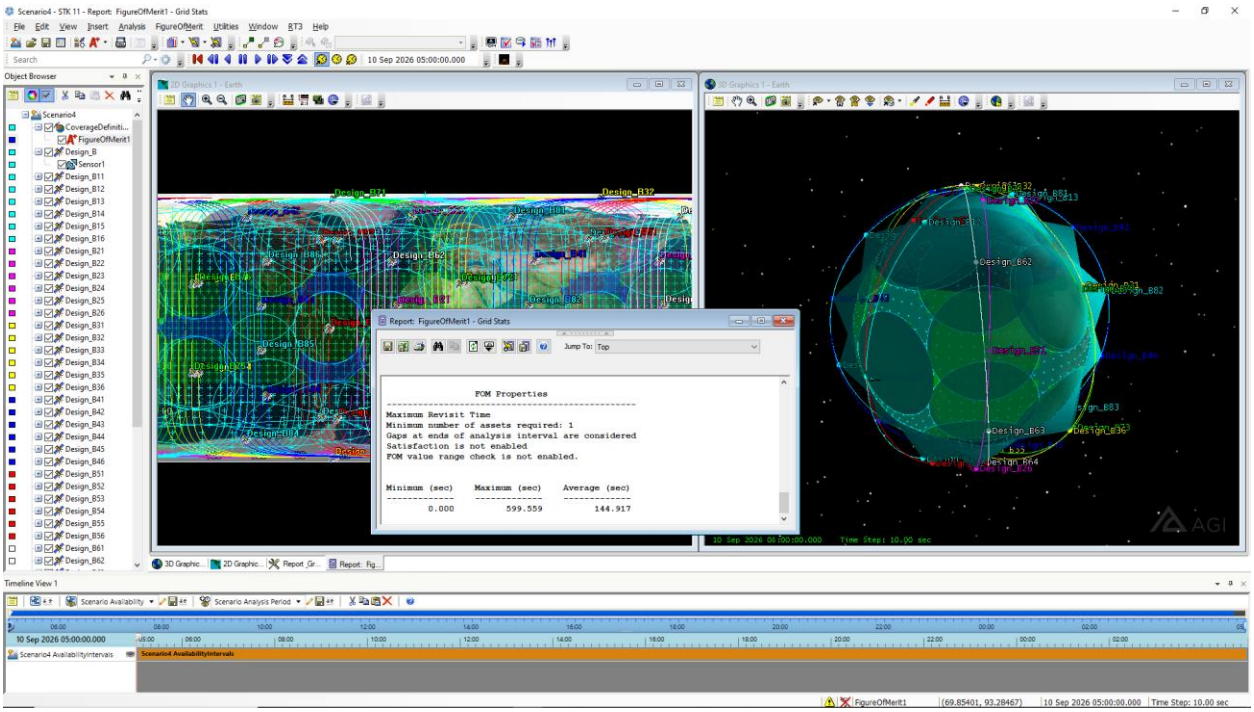
Design A:

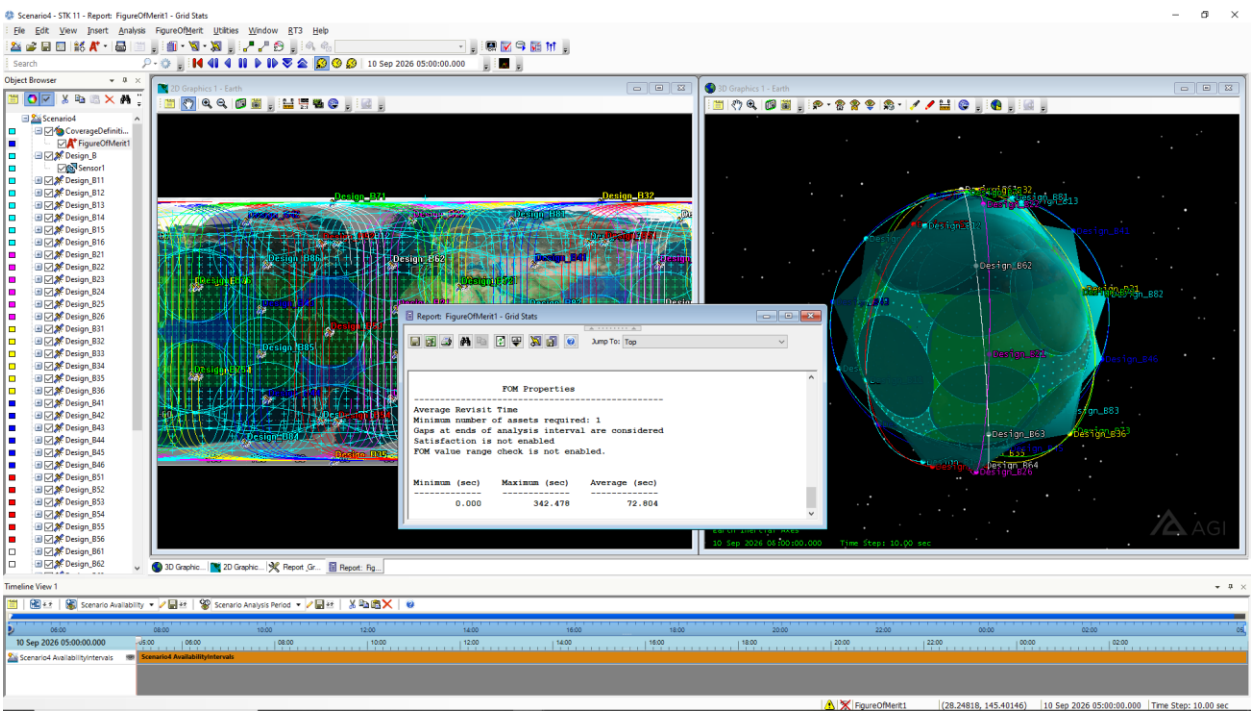


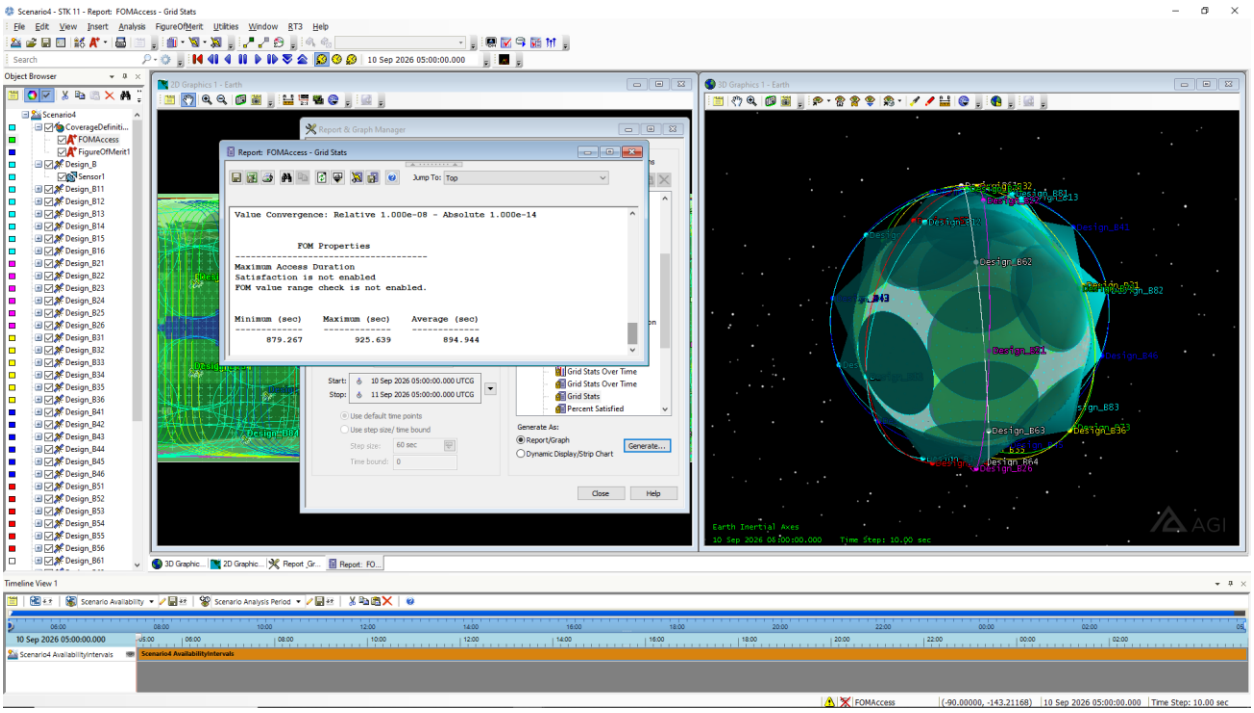

Scenario4 - STK 11 - Report: FigureOfMerit1 - Grid Stats
Report: FigureOfMerit1 - Grid Stats
Average Revisit Time
Minimum number of assets required: 1
Gaps at ends of analysis interval are considered
Satisfaction is not enabled
FOM value range check is not enabled.
Minimum (sec) Maximum (sec) Average (sec)
0.000 487.956 87.307


Scenario4 - STK 11 - Report: FOMAccess - Grid Stats
Report: FOMAccess - Grid Stats
FOM Properties
Maximum Access Duration
Satisfaction is not enabled
FOM value range check is not enabled.
Minimum (sec) Maximum (sec) Average (sec)
405.621 516.589 491.873

Design B:


Scenario4 - STK 11 - Report: FigureOfMerit1 - Grid Stats
Report: FigureOfMerit1 - Grid Stats
FOM Properties
Maximum Revisit Time
Minimum number of assets required: 1
Gaps at ends of analysis interval are considered
Satisfaction is not enabled
FOM value range check is not enabled.
Minimum (sec) Maximum (sec) Average (sec)
0.000 599.559 144.917


Scenario4 - STK 11 - Report: FigureOfMerit1 - Grid Stats
Report: FigureOfMerit1 - Grid Stats
FOM Properties
Average Revisit Time
Minimum number of assets required: 1
Gaps at ends of analysis interval are considered
Satisfaction is not enabled
FOM value range check is not enabled.
Minimum (sec) Maximum (sec) Average (sec)
0.000 342.478 72.804



Summary of Comparative Data

Metric	Design A (Python)	Design A (STK)	Design B (Python)	Design B (STK)
Altitude / Inclination	550 km / 80°	550 km / 80°	1200 km / 87°	1200 km / 87°
Constellation Pattern	120/10/1 (120 Sats)	120/10/1 (120 Sats)	48/8/2 (48 Sats)	48/8/2 (48 Sats)
Mean Revisit Time	2.4 min	1.46 min (87.31s)	1.80 min	1.21 min (72.80 s)
Worst Gap (Max Revisit)	28.00 min	27.65 min (1658.75 s)	12.00 min	9.99 min (599.56 s)
Max Access/Pass Duration	7.90 min	8.61 min (516.59 s)	14.60 min	15.43 min (925.64

Task 5: Analysis of Simulation Results

Design B Superiority: Despite deploying **60% fewer satellites** (48 vs 120), Design B achieves lower average revisit gaps (~1.21 mins) and significantly longer footprint access windows (~ 15.43 min). However, this trade-off comes with an increased edge propagation latency (10.4 ms vs 6.1 ms)